جزوهٔ درسی · جلسهٔ ۳ از ۵

مهارت‌های عامل

حافظهٔ رویه‌ای، افشای تدریجی، ارزیابی (evaluation) و آمادگی تولید

مطالعهٔ پیشنهادی: ۵۵ دقیقه **شامل:** برگهٔ تقلب عملی

فهرست مطالب

1. مقدمه: چهار اصطکاکی که حل شد
2. مهارت چیست و چطور بسازیمش
3. مهارت در برابر MCP و AGENTS.md
4. چرا این‌قدر سریع فراگیر شد
5. ارزیابی مهارت‌ها
6. از نمونهٔ اولیه تا تولید
7. فرامهارت‌ها
8. ترکیب و بسته‌بندی
9. انتخاب از میان هزاران مهارت
10. برگهٔ تقلب عملی
11. مطالعهٔ موردی: تخصص حوزه به‌مثابه کد
12. خودآزمایی

۱. مقدمه: چهار اصطکاکی که حل شد

> مهارت‌های (skill) عامل (agent) هر عامل عمومی را به‌اقتضا به متخصص بدل می‌کنند. بدون تورم زمینه (context). قابل‌حمل و سبک.

مهارت عامل (Agent Skill) راهی است برای تجهیز عامل به دانش و زمینهٔ سازمانی. از نظر ساختاری، پوشه‌ای است شامل یک فایل SKILL.md و به‌اختیار پوشه‌های scripts/ ، references/ و assets/ . مهارت‌ها

به‌سرعت به استاندارد قابلیت حمل میان‌سکویی بدل شده‌اند. چرا این‌قدر سریع؟ چون چهار نقطهٔ اصطکاک در توسعهٔ عامل را هدف می‌گیرند:

1. **دستور** (instruction) **زیاد، نتیجهٔ بدتر.** ریختن هر دستوری که به ذهنتان می‌رسد در یک پرامپت (prompt) سامانه، ناگزیر عملکرد مدل زبانی را خراب می‌کند؛ مشکلی که **پوسیدگی زمینه** (Context rot) نام دارد. مهارت‌ها این را با بارگذاری انحصاراً هنگام نیاز حل می‌کنند.
2. **دانستن «چگونه»، نه فقط «چه».** مدل‌های زبانی معادل‌های معقولی برای به‌خاطرسپردن آنچه رخ داد (حافظهٔ رویدادی) و آنچه می‌دانیم (حافظهٔ معنایی) دارند. آنچه نداشتند، راهی برای به‌خاطرسپردن **گام‌به‌گام انجام‌دادن** بود: حافظهٔ رویه‌ای (Procedural memory). مهارت‌ها را می‌توان نخستین اولیهٔ باورپذیر حافظهٔ رویه‌ای برای عامل‌ها دانست.
3. **اضافه‌بار چندعاملی.** اکوسیستم از سامانه‌های چندعاملیِ پیچیده پر شده بود که ساخت و نگه‌داری‌شان به‌شکل بدنامی دشوار است. مهارت‌ها اجازه می‌دهند یک عامل عمومی روان در نقش‌های تخصصی متعدد فرو برود.
4. **قابلیت حمل.** پوشه‌ای با یک فایل مارک‌داون، اولیه‌ای به‌طرز چشمگیری سبک است. هر عاملی که به فایل‌سیستم دسترسی دارد می‌تواند از آن استفاده کند — و همین، آن را در چشم‌اندازی با ارائه‌دهندگان متعدد کاملاً قابل‌حمل می‌سازد.

۲. مهارت چیست و چطور بسازیمش

دو مسیر پیدایش

مسیر (trajectory) **الف — ترجمهٔ آنچه از قبل می‌دانید.** متخصصان موضوعی که دانش نهادی‌شان جایی نوشته شده است: افسر انطباقی با یک دفترچهٔ سی‌صفحه‌ای، یا مدیر منابع انسانی با راهنمای آموزش تازه‌واردها. هیچ‌کدام لازم نیست برنامه‌نویسی یاد بگیرند؛ محتوا را دارند و تنها کار باقی‌مانده، ترجمهٔ آن به قالبی است که عامل هوشمندانه به کارش ببرد.

مسیر ب — تبلور آنچه عامل همین حالا انجام داد. عامل یک کار تکراری و غیرپیش‌پاافتاده را با موفقیت انجام داده است. نمی‌خواهید دفعهٔ بعد از صفر کشفش کند؛ می‌خواهید از همان اجرای موفق، مهارتی ساخته شود. الگوی در حال شکل‌گیری این است: **هر چیزی که گردش‌کار** (workflow) **خوب و قابل‌استفادهٔ مجدد باشد، مهارت می‌شود؛ و لازم نیست خودتان بنویسیدش — عامل می‌نویسد، شما بازبینی می‌کنید.**

> **اما همان معیار کیفیت برقرار است**
>
> مهارتی که عامل پیش‌نویس کرده و سپس بازبینی و اصلاح شده، می‌تواند عالی باشد. مهارتی که عامل نوشته و **بازبینی نشده**، اغلب از نبودِ مهارت هم بدتر است.

کالبدشکافی و افشای تدریجی

هر مهارت در پوشهٔ خودش قرار دارد و باید یک `SKILL.md` داشته باشد. نمونه‌ای از ساختار متعارف:

```
cafe-preparation/
├── SKILL.md                    # Required: metadata + instructions
├── scripts/                    # Optional: executable code (any language)
│   ├── calc_quantities.py      # Estimates lattes, croissants, etc.
│   └── convert_to_ingredients.py
├── references/                 # Optional: supplementary context
│   ├── menu_and_recipes.md
│   └── minimums.md
└── assets/                     # Optional: templates, configs, schemas
    ├── prep_sheet_template.md
    └── shopping_list_template.md
```

بخش نوآورانه: افشای تدریجی (Progressive disclosure)

مهارت‌ها در سه سطح بارگذاری می‌شوند:

1. **فرادداده** (نام و توصیف) همیشه در زمینهٔ عامل است.
2. **بدنهٔ** `SKILL.md` فقط وقتی بارگذاری می‌شود که مهارت فعال شود.
3. **منابع همراه** دقیقاً هنگام نیاز بارگذاری می‌شوند — و اسکریپت‌ها اجرا می‌شوند بی‌آنکه پنجرهٔ توکن (token) را آلوده کنند.

یعنی می‌توانید صد مهارت نصب‌شده داشته باشید و در هر نوبت (turn) فقط هزینهٔ ناچیز فرادادهٔ آن‌ها را بپردازید.

پیشانی‌نوشت: ماشهٔ فعال‌سازی

```
---
name: cafe-preparation
description: |
  Calculates daily ingredient needs and generates prep sheets for cafe operations
  Use when the user asks to estimate daily quantities, convert drinks to
  ingredients, or generate shopping lists.
  Do NOT use for employee shift scheduling or financial accounting.
---
```

دو چیز را باید از همان ابتدا درست کرد:

نام‌گذاری. بدیهی و کسل‌کننده باشید: برای پوشه‌ها `snake_case`، برای نام مهارت `kebab-case`، و ترجیحاً شکل مصدری مانند `managing-databases`. از نام‌های عمومی مانند `utils` یا `tools` بپرهیزید و اصطلاحات داخلی سازمان را حذف کنید.

توصیف. این، الگوریتم مسیریابی `(routing)` شماست. تنها چیزی است که مدل برای تصمیم به بارگذاری مهارت می‌بیند. بگویید چه می‌کند، کلیدواژه‌های محرک `(trigger)` را جلو بیندازید، اگر مدل کم فعال می‌کند سمج باشید، و صریح بگویید **برای چه کاری نیست**.

بقیهٔ پوشه

- **اسکریپت‌ها.** کار قطعی `(deterministic)` (تجزیهٔ خروجی، ریاضیات، قالب‌بندی) در `scripts/` قرار دارد. مدل تصمیم می‌گیرد چه کند؛ اسکریپت کار سنگین را انجام می‌دهد.
- **مراجع.** دانشی که فقط پس از فعال‌شدن مهارت مرتبط می‌شود (اصول حوزه، تعریف‌ها، رسیدگی به حالت‌های مرزی) در `references/` می‌ماند و هنگام نیاز بارگذاری می‌شود.
- **دارایی‌ها.** قالب‌ها و اسکیماها `(schema)` در `assets/` .

> **قاعدهٔ سرانگشتی**
>
> اگر `SKILL.md` دارد بلند می‌شود، پاراگراف بعدی احتمالاً جایش در `references/` است، نه در بدنه.

نصب: سه الگو

1. **انداختن فایل** (عامل‌های کدنویسی و خط فرمان): پوشهٔ مهارت را در دایرکتوری مشخصی می‌اندازید و عامل برش می‌دارد. قراردادی مشترک میان ابزارها در حال شکل‌گیری است، اما بسیاری هنوز مسیرهای اختصاصی خود را دارند. اگر میان چند ابزار `(tool)` جابه‌جا می‌شوید، مدیرهای جامعه می‌توانند کتابخانهٔ مرکزی شما را به مسیر مورد انتظار هر ابزار پیوند بدهند.
2. **نصب از رابط کاربری** (فضاهای کاری وب و سازمانی): نصب، بارگذاری و مدیریت پوشه‌های مهارت با چند کلیک از راه یک فهرست بصری، برای کل تیم.
3. **مسیر برنامه‌نویسی** (چارچوب‌های سفارشی): مهارت‌ها را برنامه‌نویسی‌شده بار می‌کنید و کد را به مسیر پوشه اشاره می‌دهید.

الگوی کلی همه‌جا یکسان است: پوشه را در دایرکتوری درست بیندازید، عامل را دوباره راه بیندازید، و آن را با یک پرامپت طبیعی بیازمایید؛ رد اجرا `(trace)` را ببینید تا مطمئن شوید مهارت واقعاً فعال شد. سپس پرامپتی بدهید که **نباید** فعالش کند و مطمئن شوید ساکت می‌ماند. آنچه میان ابزارها فرق می‌کند، «دایرکتوری درست» است.

۳. مهارت در برابر `MCP` و `AGENTS.md`

اولیه	ماهیت	نسبت با مهارت
`MCP`	دربارهٔ **دسترسی** است: سرور `MCP` عامل را به سامانهٔ بیرونی وصل می‌کند.	رقابت نمی‌کنند، ترکیب می‌شوند. مهارت دربارهٔ **دانستنِ چگونه** است: به عامل می‌آموزد چطور دربارهٔ نوع خاصی از کار فکر کند. وقتی مهارت به داده نیاز دارد، به عامل می‌گوید ابزاری را فراخوانی کند که معمولاً همان سرور `MCP` فراهمش کرده است.
`AGENTS.md`	همیشه در پروژه بارگذاری شده است.	مهارت‌ها هنگام نیاز بارگذاری می‌شوند. تمیزترین پیکربندی‌ها هر دو را دارند: `AGENTS.md` را فشرده نگه دارید (قراردادهای پروژه، پشتهٔ فناوری، فرمان‌های ساخت) و در صورت نیاز، در پایان آن فهرست کوتاهی از مهارت‌های موجود بگذارید تا نقش مسیریاب کتابخانه را هم بازی کند.

۴. چرا این‌قدر سریع فراگیر شد

تصور کنید اوایل ۲۰۲۵ است و از شما خواسته‌اند سامانه‌ای بسازید که کارهای تکراری پشتیبانی را بردارد: ساخت ارائه از روی خلاصه‌ها با رعایت سبک شرکت، تجزیهٔ فاکتورهای ساختارمند، نوشتن راهنمای آموزش منابع انسانی،

خلاصه‌کردن گزارش‌های هفتگی انطباق، و دنباله‌ای بلند از کارهای مشابه که ناگزیر رشد خواهد کرد.

به‌احتمال زیاد به‌طور پیش‌فرض سراغ معماری چندعاملی می‌رفتید: یک عامل مسیریاب در بالا و چند زیرعامل (subagent) متخصص زیر آن. سپس ساعت‌ها صرف خط لولهٔ دیپلوی، منطق هماهنگی، و اطمینان از اینکه انتشار زیرعامل تازهٔ منابع انسانی زیرعامل فاکتور را نمی‌شکند، می‌کردید.

با مهارت‌ها این گردش‌کار به‌شدت ساده می‌شود. به‌جای مسیریابی که کار را میان زیرعامل‌ها پخش می‌کند، یک عامل دارید با کتابخانه‌ای از مهارت‌ها. مهارت‌ها می‌توانند فرمان اجرا کنند، سرور MCP صدا بزنند و اسکریپت همراه داشته باشند. عامل تصمیم می‌گیرد کِی کدام را بارگذاری کند. **شما مهارت نگه‌داری می‌کنید، نه عامل — و سطح عملیاتی کوچک می‌شود.**

> **صریح بگوییم: مهارت‌ها معماری چندعاملی را نمی‌کشند**
>
> چندعاملی همچنان پاسخ کاملاً درست است وقتی موازی‌سازی واقعی دارید، مرزهای واقعی توانمندی (دسترسی متفاوت، وضعیت امنیتی متفاوت، سامانهٔ بیرونی متفاوت)، تجزیهٔ سلسله‌مراتبی که لایه‌های انتزاعش واقعاً فرق دارند، آرایش تخاصمی یا کنترل و توازن، ارتباط میان زیرعامل‌ها، یا مدل‌های ناهمگون.
>
> آنچه مهارت‌ها کردند این بود که یک **اولیهٔ معماریِ گمشده** را اضافه کردند. بسیاری از سامانه‌هایی که به‌طور پیش‌فرض چندعاملی ساخته می‌شدند، حالا می‌توانند به‌طور طراحی‌شده به «یک عامل با مهارت» ساده شوند.

مثال: شرکت لجستیکی با ۱۰۰ گونهٔ فرایند

راه‌حل	ارزیابی
یک عامل، یک کانتکست ویندوی غول‌آسا	پوسیدگی فوری زمینه و هزینهٔ توکن سرسام‌آور.
بازیابی `(retrieval)` **افزوده روی دفترچه‌ها**	دو سال پیش احتمالاً پاسخ درست بود. اما حالا یعنی نگه‌داری پایگاه بُرداری، مدل امبدینگ `(embedding)` و راهبرد تکه‌تکه‌سازی، که کیفیتشان هیچ ربطی به خود رویه‌ها ندارد.
چندعاملی، هر فرایند یک زیرعامل	صد دیپلوی، صد سطح ارزیابی و لایه‌های مسیریابی پیچیده: یک کابوس عملیاتی.
یک عامل، صد مهارت	افشای تدریجی یعنی حدود ۱۰۰ × ۵۰ ≈ ۵,۰۰۰ توکن فرادادهٔ همیشه‌بارگذاری‌شده. پرامپت‌های لجستیکی نشانه‌های فعال‌سازی قوی دارند (کد کالا، مبدأ، وزن، پرچم خطرناک، سطح خدمت) که توصیف‌ها را تیز و انتخاب را قابل‌اتکا می‌کند. رویه‌ها در کنترل نسخه‌اند و افزودن گونهٔ صدویکم یک پوشهٔ تازه است، نه یک دیپلوی `(deploy)` تازه.

با این حال مهم‌ترین نکته این است که همیشه فرایند ارزیابی سخت‌گیرانه‌ای داشته باشید و عملکردها را برای تصمیم نهایی مقایسه کنید. و یک نکتهٔ جامعه‌شناختی: **پذیرش از مسیر کم‌مقاومت‌ترین راه می‌آید. هرکسی که بتواند مستند بنویسد، می‌تواند مهارت بنویسد.** همین سد ورود را پایین می‌آورد و دانش رویه‌ای نهفته در ویکی‌ها، دفترچه‌ها و ذهن مهندسان، بالاخره جایی ساختارمند برای جاری‌شدن پیدا می‌کند.

۵. ارزیابی مهارت‌ها

> مهارتی که تست `(test)` ندارد، یک قابلیت نیست؛ یک امید است.

عدد هشداردهنده

در بنچمارکی روی ۸۴ کار واقعی، پژوهشگران دریافتند **۱۹٪ موارد با مهارت بدتر از بدون مهارت** عمل کردند. این مهارت‌های بدطراحی فقط نویز خنثی نبودند؛ فعالانه توانایی را کم کردند.

چهار حالت شکست

1. **شکست محرک** (Trigger failure)**:** مهارت اشتباه فعال می‌شود، یا مهارت درست فعال نمی‌شود. **کجا دیده می‌شود: گزارش‌های مسیریابی.**
2. **شکست اجرا:** مهارت درست فعال می‌شود اما خروجی نادرست یا فراخوان ابزار (tool call) خطا تولید می‌کند. **کجا: کیفیت خروجی.**
3. **شکست بودجهٔ توکن** (token budget)**:** بدنهٔ حجیم، کانتکست ویندو (context window) را پر می‌کند و عملکرد را در نوبت‌های بی‌ارتباط خراب می‌کند. **کجا: زیر بار واقعی زمینه.**
4. **پس‌رفت** (regression)**:** مهارت تازه با مهارت موجود هم‌پوشانی پیدا می‌کند و مسیریابی درستِ قبلی را می‌شکند. **کجا: فقط وقتی کل کتابخانه با هم آزموده شود.**

دو حالت اول در سطح تک‌نوبتی رخ می‌دهند؛ دو حالت آخر فقط وقتی ظاهر می‌شوند که چند مهارت زیر بار واقعی تولید با هم تعامل کنند.

جعبه‌ابزار ارزیابی

الگو	شرح	حالت شکستِ هدف	چه زمانی لازم است
ارزیابی به‌مثابه تست واحد	فایل تستی که در هر تغییر روی سی‌آی (CI) اجرا می‌شود؛ تست مردود، ادغام را مسدود می‌کند.	همه	هر مهارت، هر تغییر
مجموعه‌دادهٔ طلایی	جفت‌های گزینش‌شده و نسخه‌بندی‌شدهٔ ورودی و خروجی مورد انتظار، ذخیره‌شده کنار خود مهارت.	اجرا، محرک	از سطح پیش‌نویس به بالا
داور مدل‌زبانی (LLM-as-Judge)	مدلی هم‌رده که خروجی را در مقیاس با سنجه (rubric) نمره می‌دهد؛ با جابه‌جایی موقعیت‌ها برای خنثی‌کردن سوگیری ترتیب.	اجرا	پیش از رسیدن به سطح کنش
تخاصمی / تیم قرمز	کاوش نظام‌مند برای بیرون‌کشیدن شکست‌ها: برای هر محرک مثبت، یک بازنویسی و یک حالت مرزی (edge case) منفی.	محرک، اجرا	پیش از هر انتشار در سطح کنش
قناری / سایه	دیپلوی روی ترافیک کنترل‌شده پیش از انتشار کامل: مقایسهٔ موازی برون‌خط، یا درصد کوچکی از ترافیک زنده زیر پایش.	پس‌رفت	پیش از انتشار کامل

محرک، نخستین دروازه است

مهارتی که هرگز فعال نمی‌شود نمی‌تواند کمک کند؛ مهارتی که بیش‌ازحد گسترده فعال می‌شود زمینهٔ نامربوط تزریق می‌کند. یک تحلیل تولیدی نرخ عدم فعال‌شدن **۵۶٪** را برای مهارت‌هایی گزارش کرد که انتظار می‌رفت پیوسته فعال شوند. مهم‌تر اینکه مهارتی که دستورهایش حذف شده بود امتیاز ۵۸ گرفت، در حالی که عامل بدون مهارت ۶۳ گرفت: این کسری پنج‌واحدی نشان می‌دهد مهارت بدطراحی می‌تواند فعالانه توانایی را کم کند.

نکتهٔ مکمل همان مطالعه

یک فهرست **منفعل** از قراردادهای پروژه در `AGENTS.md` نرخ قبولی ۱۰۰٪ در برابر پایهٔ ۵۳٪ گرفت. این تأکید می‌کند که مهارت‌ها بهتر است برای گردش‌کارهای **باریک و کنش‌محور** کنار گذاشته شوند، در حالی که زمینهٔ سراسری باید در مستنداتِ منفعل و همیشه‌دردسترس بماند.

برای رسیدن به دقت محرک در حد استاندارد صنعت، توصیف باید چهار وارسی را پاس کند:

- **ویژگی تست‌پذیر:** باید بتوانید سه محرک مثبت و سه محرک منفی بنویسید.
- **روشنی:** پرسش‌های مبهم با مهارت‌های همسایه هم‌پوشانی پیدا نکنند.
- **صداقت اجرایی:** عملکرد واقعی را توصیف کند، نه رفتار آرمانی.
- **پایداری در بازنویسی:** مستقل از شیوهٔ جمله‌بندی کاربر، یکسان مسیریابی کند.

کیفیت خروجی و مسیر ابزار

وقتی مهارت فعال شد، خروجی نهایی (آنچه عامل می‌گوید) و مسیر ابزار `(Tool trajectory)` (آنچه عامل انجام می‌دهد) را **جداگانه** بیازمایید. تحلیل‌ها نشان می‌دهند نمره‌دهی فقط بر خروجی نهایی، ۲۰ تا ۴۰ درصد بیشتر از نمره‌دهی مسیرآگاه قبول می‌کند؛ این شکاف یعنی مواردی که عامل با توالی نادرست فراخوان ابزار به پاسخ درست رسیده است. در سناریوهای فقط‌خواندنی پذیرفتنی است؛ در مهارت‌های مجاز به کنش، بحرانی — چون مسیر نادرست می‌تواند عوارض جانبی برگشت‌ناپذیر بسازد. اعتبارسنجی مسیر باید با سطح مهارت هم‌راستا باشد: مهارت‌های فقط‌خواندنی می‌توانند ترتیب دلخواه را بپذیرند؛ مهارت‌های مجاز به کنش به ترتیب دقیق یا زیرمجموعهٔ مرتب نیاز دارند.

توسعهٔ ارزیابی‌محور

گردش‌کار را وارونه کنید: **پیش از** نوشتن `SKILL.md` ، سه پروندهٔ ارزیابی JSON بنویسید (ورودی، ابزارهای مورد انتظار، خروجی مورد انتظار). این کار مشخصات `(spec)` کارکردی روشنی را از ابتدا تحمیل می‌کند:

```
{
  "case_id": "refund_dup_charge_001",
  "input": "I was charged twice for order #4521 last Tuesday",
  "expected_skill": "refund_processor",
  "expected_tool_calls": [
    {"tool": "lookup_order", "args": {"order_id": "4521"}},
    {"tool": "check_duplicate_charge", "args": {"order_id": "4521"}}
  ],
  "expected_output_format": "confirmation_with_refund_id",
  "rubric": ["acknowledges duplicate", "cites order id", "provides next step"]
}
```

نوشتن سه پروندهٔ این‌چنینی از ابتدا، ابهام‌های توصیف و خطاهای مسیر ابزار را پیش از آنکه در بدنهٔ مهارت انباشته شوند بیرون می‌کشد. و هنگام استفاده از داور `judge` مدل‌زبانی دو چیز غیرقابل‌مذاکره است: **جابه‌جاکردن موقعیت** خروجی مرجع و خروجی واقعی برای حذف سوگیری ترتیب، و **کالیبراسیون** در برابر ارزیابی انسانی تا رسیدن به توافق بالا.

بودجهٔ توکن: انزوا یک دام است

هرگز مهارت را صرفاً در انزوا ارزیابی نکنید. عامل‌ها در تولید پنج تا پانزده مهارت را هم‌زمان بارگذاری می‌کنند. بدنه‌ای که از پنج هزار توکن عبور می‌کند ممکن است تنها بی‌نقص کار کند، اما هم‌بارگذاری‌شده پوسیدگی زمینه می‌سازد. پژوهش‌ها افت دقت چشمگیر ناشی از تکثیر ابزار و رقابت توجه، و نیز افت همهٔ مدل‌های پیشرو با بزرگ‌شدن ورودی را ثبت کرده‌اند.

دام ارزیابی مرکب: مهارت یا عامل؟

تست مسیر، سامانهٔ مرکب «مهارت + عامل میزبان» را می‌سنجد. اگر تستی مردود شد، از بیش‌مهندسی `SKILL.md` برای یک مدل خاص بپرهیزید، چون قابلیت حمل را نابود می‌کند. به‌جایش منطق اجرا را از مسیریابی جدا کنید: کد ابزار زیرین را مستقل اعتبارسنجی کنید و محرک‌ها را روی چند خانوادهٔ مدل ممیزی کنید تا توصیف‌های شکننده و قفل‌شده به یک معماری را بگیرید. برای کالیبراسیون ساده‌تر، از الگوی «زیرعامل تک‌مهارتی» استفاده کنید (عامل + یک مهارت در برابر عامل پایه).

نردبان اختیار

سطح	شرط عبور
فقط‌خواندنی	ارزیابی با داور مدل‌زبانی؛ دقت محرک در حد هدف.
فقط‌پیش‌نویس (بازبینی انسانی)	مجموعه‌دادهٔ طلایی با ۲۰ پرونده یا بیشتر؛ تأیید انسانی.
مجاز به کنش	تیم قرمز کامل؛ موفقیت پایدار در چند اجرا؛ بدون رویداد رول‌بک؛ `pass^k` پایدار.

سنجهٔ `pass^k` موفقیت **پیوسته** را می‌سنجد نه **گاه‌به‌گاه**: ارزیابی را `k` بار اجرا کنید و موفقیت در همهٔ اجراها را لازم بدانید. در یک بنچمارک (`benchmark`) شناخته‌شده، مدلی که در اجرای نخست ۶۱٪ گرفت، در هشت اجرای متوالی به زیر ۲۵٪ سقوط کرد.

> **دو عامل مهم در کالیبراسیون آستانه‌ها**
>
> **افت تولید:** عملکرد تولیدی معمولاً ۲۰ تا ۳۰ درصد پایین‌تر از اعداد بنچمارک برون‌خط است. **سوگیری شبیه‌سازی:** ارزیابی‌های مبتنی بر شبیه‌سازی می‌توانند تا حدود ۹ درصد خوش‌بینانه باشند. در نتیجه، بازبینی انسانی نمونه‌های نماینده همچنان نهایی‌ترین سیگنال اعتبارسنجی برای عبور به سطح کنش است.

«پوشش ارزیابی» یعنی چه

مهارت وقتی پوشش (`coverage`) ارزیابی کامل دارد که هر چهار شرط برقرار باشد:

- **محرک:** راستی‌آزمایی رفتار فعال‌سازی با پرونده‌های مثبت (باید فعال شود) و منفی (نباید فعال شود).
- **اجرا:** خروجی درست در گسترهٔ نماینده‌ای از ورودی‌های مورد انتظار.
- **پس‌رفت:** افزودن این مهارت هیچ افتی در کتابخانهٔ موجود ایجاد نکند.
- **بودجهٔ توکن:** ردپای توکنی مهارت کران‌دار باشد تا عملکرد نوبت‌های بی‌ارتباط را خراب نکند.

رد شدن در هر یک از این چهار شرط، مهارت را در سطح پیش‌نویس نگه می‌دارد، صرف‌نظر از عملکردش در مسیر خوش‌بینانه.

۶. از نمونهٔ اولیه تا تولید

داخل رانتایم عامل چه هست؟

زیر پوستهٔ چارچوب‌ها، حلقهٔ عامل میان ارائه‌دهندگان هم‌گرا شده است: رانتایم (runtime) گفت‌وگو را نگه می‌دارد، مدل را صدا می‌زند، ابزار اجرا می‌کند، فایل می‌خواند و پاسخ برمی‌گرداند. آنچه در بررسی این زمان‌های اجرا چشمگیر است، این است که چقدر کمی از کد به استدلال مربوط می‌شود. مهندسی معکوس اخیر یک عامل کدنویسی شناخته‌شده نشان داد **۹۸٫۴٪ کدبیس زیرساخت عملیاتی است** — دسته‌بندهای مجوز، خطوط لولهٔ فشرده‌سازی زمینه، واگذاری (handoff) به زیرعامل، ذخیره‌سازی نشست (session) — و تنها ۱٫۶٪ خودِ حلقهٔ عامل.

> با هم‌گرایی مدل‌های پایه در استدلال پایه، تمایزدهندهٔ اتکاپذیری خودگردان، مهندسی قطعیِ پیرامون مدل می‌شود؛ و درون آن مهندسی، واحدی که ترکیب و بازاستفاده می‌شود، مهارت است.

چرا مهارت واحد بهبود است

نظریهٔ ساده‌انگارانه می‌گوید مدل‌های بهتر عامل‌های بهتر می‌سازند. در تولید، مدل زیرساخت است و مهارت‌ها اولیه‌ای‌اند که اجازه می‌دهند بهبودها منتشر شوند. هر مهارت تازه یک واحد کوچک، مالک‌دار و تست‌پذیر از توانایی است. وقتی حالت مرزی تازه‌ای ظاهر شود، فقط یک `SKILL.md` ویرایش می‌شود و توانایی مؤثر عامل بدون دردسرهای مهندسی پرامپتِ یکپارچه رشد می‌کند. سه ویژگی این را ممکن می‌کند:

- **مشروط‌اند:** فقط وقتی بارگذاری می‌شوند که توصیفشان با کار بخواند.
- **ترکیب‌پذیرند:** یک مهارت می‌تواند ابزار مهارت دیگر را فراخوانی کند یا در زنجیره قرار بگیرد، بی‌آنکه هیچ‌کدام از دیگری خبر داشته باشد.
- **مالک دارند:** هرکدام در پوشه‌ای نسخه‌بندی‌شده با نویسندهٔ مشخص قرار دارد؛ پس بهبود توزیع می‌شود و در تیم سکوی مرکزی گلوگاه نمی‌سازد.

سبک بهبود	زمان چرخه	حالت شکست و هزینهٔ زمینه	چه کسی می‌تواند
تعویض مدل	روزها تا هفته‌ها	پس‌رفت در کارهای بی‌ارتباط (هزینهٔ زمینه: ندارد)	تیم یادگیری ماشین یا سکو
ویرایش سیستم پرامپت	دقیقه‌ها تا ساعت‌ها	پوسیدگی زمینه، تعارض دستورها (هزینهٔ زمینه: ایستا — هر نوبت)	مالک فایل پرامپت
فاین‌تیون (fine-tuning)	هفته‌ها تا ماه‌ها	فراموشی فاجعه‌بار، بیش‌برازش (هزینهٔ زمینه: ندارد)	فقط تیم یادگیری ماشین
مهارت تازه	ساعت‌ها تا روزها	کران‌دار: فقط نوبت‌های متناظر (هزینهٔ زمینه: پویا (dynamic) — هنگام فعال‌شدن)	هر تیم حوزه‌ای

سرریز زمینه: حالت شکستی که نمایش‌ها را می‌شکند

شایع‌ترین حالت شکست عامل‌ها در تولید توهم (hallucination) نیست؛ **سرریز زمینه** است: مدل زمینه‌ای بیش از آنچه بتواند مؤثر به کار ببرد دریافت می‌کند و بی‌صدا افت می‌کند، پیش از آنکه اپراتور متوجه شود. دو رشته پژوهش این را پشتیبانی می‌کنند: **«گم‌شدن در میانه** (Lost in the middle)» نشان می‌دهد عملکرد وقتی بیشترین است که اطلاعات مرتبط در ابتدا یا انتهای ورودی باشد و در میانه افت می‌کند — منحنی‌ای که حتی برای مدل‌های آموزش‌دیده روی زمینهٔ بلند هم برقرار است؛ و **«پوسیدگی زمینه»** در بررسی ۱۸ مدل پیشرو نشان می‌دهد عملکرد با بزرگ‌شدن ورودی افت می‌کند، حتی وقتی دشواری کار ثابت نگه داشته شود. و نویز معمول زمینهٔ واقعی عامل‌ها — خروجی ابزارها، بازیابی‌های نیمه‌مرتبط و استدلال‌های میانی — از بدترین انواع نویز است.

ریاضیات بودجهٔ توکن

عاملی با پنجاه گردش‌کار متمایز را در نظر بگیرید. به‌شکل یک سیستم پرامپت، در هر نوبت ۱۵٬۰۰۰ توکن بارگذاری می‌شود. به‌شکل کتابخانهٔ مهارت، حدود ۴٬۰۰۰ توکن توصیف به‌علاوهٔ بدنهٔ حدوداً ۲٬۰۰۰ توکنی تنها مهارت فعال بارگذاری می‌شود: مجموعاً حدود ۶٬۰۰۰ توکن، و چهل‌ونُه بدنهٔ دیگر روی دیسک می‌مانند. نمونه‌های منتشرشده‌ای وجود دارد که در آن‌ها تبدیل یک گردش‌کار به مهارت، زمینهٔ فعال را از حدود ۱۵۰٬۰۰۰ توکن به ۲٬۰۰۰ توکن رسانده است؛ بیش از ۹۸ درصد کاهش.

سه پیامد عملی:

- **ظرفیت سنجهٔ اشتباهی است.** پنجره‌ای با ظرفیت یک میلیون توکن می‌تواند در پنجاه هزار توکن افت معناداری نشان دهد.

- **زمینهٔ فعال یک بودجه است، نه یک ظرف.** هر توکن پیش روی مدل، توجه را از هر توکن دیگر می‌گیرد. با سیستم پرامپت همان‌طور رفتار کنید که تیم زیرساخت با حافظه (memory): منبعی متناهی که آگاهانه تخصیص می‌یابد.
- **مهارت‌ها قید (constraint) را حل می‌کنند.** زمینهٔ فعال را کوچک نگه می‌دارند، در حالی که توانایی در دسترس عملاً بی‌کران (Unbounded) می‌ماند.

۷. فرامهارت‌ها

تا اینجا هر مهارت را انسان نوشته است و این نقطهٔ شروع درستی است. اما وقتی کتابخانه‌ای کارآمد دارید، پرسش طبیعی بعدی این است: آیا عامل می‌تواند در نوشتن، ارزیابی و بهبود مهارت‌ها هم کمک کند؟ فرامهارت‌ها (Meta-skill) در چهار دسته جای می‌گیرند:

1. **نگارش:** مهارتی که توصیف یک گردش‌کار را می‌گیرد و پیش‌نویس `SKILL.md` تولید می‌کند.
2. **نگارش کمکی از روی رد اجرا:** به‌جای آنکه از انسان بخواهید گردش‌کار را توصیف کند، عامل را چند بار در انجام موفق آن تماشا کنید و همان رد را به مهارت تبدیل کنید. کار انسان از نوشتن مهارت به تأیید اینکه نسخهٔ برداشت‌شده گام‌های درست را گرفته، جابه‌جا می‌شود.
3. **بهبود:** مهارتی که یک مهارت موجود به‌همراه مجموعه‌ای از پرونده‌های ارزیابی مردود را می‌گیرد و ویرایش پیشنهاد می‌دهد؛ الگویی که در آن عامل تغییری پیشنهاد می‌کند، آزمایشی کران‌دار اجرا می‌کند، و تغییر را فقط در صورت بهبود سنجه نگه می‌دارد.
4. **تکامل کتابخانه:** مهارت‌هایی که کتابخانه را در طول زمان رشد می‌دهند: عامل کاری را تمام می‌کند که مهارتی برایش نداشت، متوجه می‌شود مسئله‌ای تکرارشونده را حل کرده، و افزودن مهارتی تازه را پیشنهاد می‌دهد.

> **کجا از هم می‌پاشد**
>
> فرامهارت‌ها فقط وقتی کار می‌کنند که مجموعهٔ ارزیابی شما خوب باشد. عاملی که اجازه دارد مهارت‌های خودش را ویرایش کند، با کمال میل برای هر سنجه‌ای که به آن اشاره کنید بهینه می‌شود، از جمله سنجه‌هایی که به‌آسانی می‌توان سرشان کلاه گذاشت. بدون تست‌های محکمِ دقت محرک، تست‌های پس‌رفت و وارسی‌های انسانی، یک حلقهٔ بهبود خودگردان (autonomous) بی‌سروصدا کتابخانهٔ شما را بدتر می‌کند، در حالی که گزارش می‌دهد دارد بهتر می‌شود.

سه عادت که دوام آورده‌اند:

- هر چیزی که عامل می‌نویسد، در **سطح پیش‌نویس** وارد کتابخانه می‌شود، هرقدر هم فرامهارت مطمئن باشد؛ و از همان نردبان مهارت‌های انسان‌نوشته بالا می‌رود.

- برای چند ویرایش نخست **انسان را در حلقه** (loop) **نگه دارید**. حتی وقتی سنجه آشکارا بهتر شده، تفاوت‌ها را اسکن کنید. خطای عامل — بیش‌برازش توصیف به چند پروندهٔ تست، یا خراب‌کردن مهارتی که از وجودش خبر نداشت — دقیقاً همان نوعی است که انسان در سی ثانیه می‌گیرد.
- **با فرامهارت شروع نکنید.** اول حلقهٔ نگارش دستی را راه بیندازید. سریع‌ترین راه برای ساختن یک کتابخانهٔ بد این است که عامل را به پوشه‌ای خالی اشاره کنید و بخواهید پنجاه مهارت بسازد.

۸. ترکیب و بسته‌بندی مهارت‌ها

گردش‌کارهای واقعی درون یک مهارت جا نمی‌شوند. مسئلهٔ ترکیب این است که مهارت‌ها چطور به هم ارجاع بدهند، وضعیت را رد و بدل کنند و از وابستگی‌های (dependency) حلقوی بپرهیزند. رد و بدل خام خروجی مدل میان مهارت‌های منزوی در سامانه‌ای یکپارچه کارساز نیست: وضعیت مبهم می‌شود، اجرا غیرقطعی (not deterministic) می‌شود و دیباگ (debug) سخت.

مسیریابی اجرا: هماهنگی گراف بدون دور

- **وضعیت جداشده:** مسیریابی وضعیت به انباشتن تاریخچهٔ اجرا در پرامپت مدل تکیه ندارد.
- **گذرگاه پیام فایلی:** کنترل‌گر گراف تحویل‌ها را با رد و بدل ارجاع‌های اسکیمای‌ای میان گره‌ها هماهنگ می‌کند.
- **توجه محافظت‌شده:** جداکردن بار داده از ورودی متنی مدل از تورم کانتکست ویندو جلوگیری می‌کند و ظرفیت مدل را حفظ می‌کند.

بسته‌بندی محیط: نمایه‌های توانمندی

فعال‌کردن همهٔ مهارت‌ها، مسیریابی زبان طبیعی را خراب و کانتکست ویندو را اشباع می‌کند. به‌جایش از «نمایه‌های توانمندی» استفاده کنید: شخصیت‌های تخصصی متناسب با وضعیت‌های اجرایی مشخص. هر نمایه یک بستهٔ ماژولار است که مهارت‌های فعال و دسترسی ابزارها، سیستم پرامپت‌ها و گاردریل‌ها، گردش‌کارهای خودکار و آرایش زیرعامل‌ها، و پارامترهای مدل را تعریف می‌کند. هنگام اجرا، لایهٔ هماهنگی دستورهای پیشین را تخلیه و متغیرهای کهنه را پاک می‌کند و سپس نمایهٔ تازه را جایگزین می‌کند؛ این فرایند سخت‌گیرانهٔ برچیدن و بازسازی، از گم‌شدن زمینه جلوگیری می‌کند.

گونه‌شناسی متعارف مهارت‌ها

- **مولد:** تبدیل قصد (intent) کاربر به مصنوع ساختارمند.
- **بازبین و دروازه** (gate)**:** دروازه‌های قطعی که در صورت ناموفق‌بودن اعتبارسنجی، اجرا را مسدود می‌کنند.
- **خط لوله:** هماهنگی مسیرهای خطی درون محیط گراف بزرگ‌تر.

- **وارونگی و بازیابی:** وادارکردن عامل به شفاف‌سازی پیش‌فرض‌ها پیش از اجرا.
- **پوشش‌های زمینهٔ حوزه:** گره‌های مرجعی که قراردادهای حوزه را می‌آموزند.

بدهی زمینه و انتقال هوش به چپ

مهارت‌ها توجه مدل را می‌سوزانند و توجه منبعی کمیاب است. وقتی نویسندگان می‌کوشند با متورم‌کردن توصیف‌ها (مثلاً «همیشه فلان کار را بکن» با حروف بزرگ) رفتار قطعی در رانتایم بسازند، **بدهی زمینه** (Context debt) انباشته می‌شود. مدل‌ها یاد می‌گیرند این دستورهای پرتأکید را نادیده بگیرند، دقیقاً همان‌طور که یک توسعه‌دهندهٔ انسانی دیوار هشدارهای ناخوانا را نادیده می‌گیرد.

بهترین روش‌های عملی

نرم‌افزار بنویسید، نه قاعده: دستورهای منفی به مدل را با قیدهای نرم‌افزاری قطعی جایگزین کنید که کنش نامعتبر را ناممکن می‌کنند.

افشای تدریجی را پیاده کنید: دستورهای پیچیده را فقط هنگام فراخوانی صریح مهارت بارگذاری کنید.

وضعیت را جدا کنید: هرگز از کانتکست ویندو به‌عنوان پایگاه داده استفاده نکنید؛ فقط نشانی یا اشاره‌گر را از راه فایل‌سیستم یا گذرگاه پیام رد کنید.

۹. انتخاب از میان هزاران مهارت

تا اوایل ۲۰۲۶، مارکت‌پلیس‌های (marketplace) عمومی مهارت از ده‌ها هزار فهرست عبور کرده بودند و ریپوهای رسمی ارائه‌دهندگان نیز راه‌اندازی شدند. مسئلهٔ انتخاب واقعی و رو به رشد است. سه معیار کمک می‌کنند:

1. **برای ابزارهای اختصاصی، مهارت دست‌اول را ترجیح دهید.** هر چیزی که نویسندگانِ خودِ سامانهٔ زیرین نوشته باشند، درست‌تر و نگه‌داشته‌شده‌تر از جایگزین‌های جامعه خواهد بود.
2. **هر چیزی را که به آن وابسته‌اید قفل نسخه کنید.** مهارت‌های جامعه تکامل می‌یابند و وابستگی قفل‌نشده‌ای که دیروز کار می‌کرد فردا خراب می‌شود.
3. **پیش از پذیرش ممیزی کنید.** مهارت، کدی است که در زمینهٔ شما اجرا می‌شود. با آن مثل هر وابستگی دیگری رفتار کنید، با همان بهداشت زنجیرهٔ تأمین.

منبع	پیش‌فرض اعتماد	چه کسی نگه‌داری می‌کند
مهارت‌های دست‌اول ارائه‌دهنده	اعتماد پیش‌فرض؛ نسخه را قفل کنید	تیمی که محصول زیرین را ساخته است
مهارت‌های گزینش‌شدهٔ سازمان	اعتماد درون سازمان؛ بازبینی هنگام پذیرش	تیم‌های حوزه‌ای خودتان، با بازبینی در پول‌ریکوئست `(pull request)`
مهارت‌های جامعه	ممیزی پیش از پذیرش؛ قفل نسخهٔ سخت‌گیرانه	نویسندگان داوطلب با تعهد نگه‌داری متغیر

۱۰. برگهٔ تقلب عملی

حداقل `SKILL.md`

```
---
name: skill-name
description: |
  [What it does in one verb-led sentence.] Use this skill when the user
  [trigger phrase 1], [trigger phrase 2], or [trigger phrase 3].
  Do NOT use for [anti-trigger 1] or [anti-trigger 2].
version: 1.0.0
license: MIT
allowed-tools: [Optional] Read Bash Write
---

# Skill Name

## When to use
- [Concrete scenario]

## When NOT to use
- [Out-of-scope scenario]

## Workflow
1. [Step]
2. [Step]
3. See `references/advanced.md` for [edge case].

## Examples
- Input: "..."  →  Output: "..."

## Output format
- Use `assets/template.md`

## Anti-patterns to avoid
- Don't [...]
```

پنج قاعده

1. **یک مهارت، یک کار.** اگر نمی‌توانید کارِ مهارت را در یک جمله بگویید، دو مهارت است. پیش از نوشتن تجزیه (decomposition) کنید.
2. **توصیف یک رابط است.** عامل مهارت‌ها را با خواندن توصیف انتخاب می‌کند. توصیف مبهم یعنی مهارت بلااستفاده.
3. **مهارت‌ها وابستگی‌اند.** مثل کتابخانه‌ها با آن‌ها رفتار کنید: نسخه‌بندی، قفل نسخه، بازبینی در پول‌ریکوئست.
4. **تیم درست، مالک مهارت درست.** متخصصان حوزه مالک مهارت‌های حوزه‌اند. نگذارید تیم هوش مصنوعی گلوگاه دانش حوزه‌ای سازمان شود.
5. **رانتایم عامل قابل‌تعویض است.** مهارت‌ها را به یک رانتایم گره نزنید؛ قابلیت حمل بخشی از ارزش است.

شش اصل کیفیت

1. **اول خودتان کار را انجام دهید.** خطای واقعی سیگنال تولید می‌کند؛ گمانه‌زنی نویز.
2. **دلیل را بگویید، نه فقط قاعده را.** مدل‌ها وقتی بفهمند چرا دستوری وجود دارد، به‌خوبی به حالت‌های مرزی تعمیم می‌دهند. اگر دیدید دارید «همیشه» و «هرگز» با حروف بزرگ می‌نویسید، مکث کنید و به‌جایش منطق را توضیح دهید.
3. **هر سطر باید جای خودش را بخرد.** نکته‌های ظریف، فرمان‌های دقیق، منطق کسب‌وکار و ضدالگوها را نگه دارید؛ چیزهایی را که مدل از قبل می‌داند حذف کنید.
4. **یک مهارت، یک کار.** اگر توصیف میان دو توانایی بی‌ارتباط نیاز به «و» دارد، تقسیمش کنید.
5. **دستورها را راستی‌آزمایی‌پذیر کنید.** اگر عامل نتواند بفهمد قاعده را رعایت کرده یا نه، قاعده بیش‌ازحد مبهم است.
6. **آنچه تکرار می‌شود را بسته‌بندی کنید.** کد کمکی‌ای که عامل مدام از نو می‌سازد، جایش در `scripts/` است.

بوی بد مهارت (اگر دیدید، بازنویسی کنید)

- بیش از پنج هزار واژه: احتمالاً دو مهارت است، یا مطلب مرجعی که باید در `references/` باشد.
- دو تیم حوزه‌ای می‌توانند به‌طور موجه مالکش باشند: هنوز تجزیه نشده؛ در مرز تیم‌ها تقسیمش کنید.
- نمی‌توانید برایش سه پروندهٔ تست بنویسید: توصیف بیش‌ازحد مبهم است و مهارت کارهای زیادی می‌کند.
- به هیچ منبع دیگری ارجاع نمی‌دهد: شاید فقط یک دستور بلند است که جایش در سیستم پرامپت بوده.
- مدام بخش «حالت مرزی» اضافه می‌کنید: هر حالت مرزی احتمالاً مهارت خودش را می‌خواهد.

- توصیفش با «مهارتی مفید برای...» شروع می‌شود: بازنویسی کنید؛ توصیف باید محرک، ورودی و خروجی را نام ببرد.

سیاههٔ وارسی پوشش ارزیابی

- ☐ **محرک:** پرونده‌های مثبت و منفی؛ هدف، دقت محرک در حد استاندارد.
- ☐ **اجرا:** خروجی درست در گسترهٔ نماینده‌ای از ورودی‌ها.
- ☐ **پس‌رفت:** افزودن این مهارت هیچ افتی در مجموعهٔ کتابخانهٔ موجود نسازد.
- ☐ **بودجهٔ توکن:** هم‌بارگذاری‌شده با ۵ تا ۱۵ مهارت پرکاربرد، نوبت‌های بی‌ارتباط را خراب نکند.

سیاههٔ وارسی دیپلوی

- ☐ پیشانی‌نوشت معتبر است (وارسی نحوی پاس می‌شود).
- ☐ توصیف شامل «چه می‌کند» + «کِی» + «کِی نه» است.
- ☐ اسکریپت‌ها تست واحد دارند و در سی‌آی قبول می‌شوند.
- ☐ مجموعهٔ ارزیابی با آستانهٔ حداقلی قبول می‌شود.
- ☐ پویش امنیتی تمیز است (بدون راز، بدون وابستگی نامطمئن).
- ☐ توصیف را کسی جز نویسنده بازبینی کرده است.
- ☐ اگر عمومی منتشر می‌شود، مسیرهای نصب چندابزاری آزموده شده‌اند.

مدل ذهنی یک‌خطی

سیستم پرامپت = غریزه · `AGENTS.md` = فایل راهنمای پروژه · ابزارها و `MCP` = دست‌ها · بازیابی = کتابخانه · **مهارت‌ها = دفترچهٔ راهنمایی که همکار باتجربه روز اول به شما می‌دهد، و هوش مصنوعی هرگز فراموشش نمی‌کند.**

۱۱. مطالعهٔ موردی: تخصص حوزه به‌مثابه کد

دو خرده‌فروش که روی رانتایم یکسانی کار می‌کنند و از راه رابط‌های مشابه به دادهٔ مشابه دسترسی دارند، تجربه‌های خرید کاملاً متفاوتی تولید می‌کنند. تخصص خرده‌فروشی از آن دانش‌هایی است که تاریخاً در سه جای دسترس‌ناپذیر گیر کرده بود: در ذهن خریداران ارشد و مدیران دسته، در دفترچه‌های عملیاتی سی‌صفحه‌ای که کسی نمی‌خواندشان،

و در رشته‌گفت‌وگوهای کاری از سال‌ها پیش. مهارت‌ها این دانش را به شکلی می‌گیرند که سامانه‌های مشتری‌محور شرکت واقعاً به کارش می‌برند.

سه لایهٔ معماری

- **سطح مشتری:** گفت‌وگو در وب‌سایت، برنامهٔ موبایل، کیوسک فروشگاه، عامل صوتی مرکز تماس. هر سطح نازک است: ورودی را به رانتایم می‌فرستد و پاسخ را نمایش می‌دهد.
- **رانتایم و هماهنگ‌کننده** `(Orchestrator)`**:** گفت‌وگو را نگه می‌دارد، مهارت‌ها را بارگذاری می‌کند، ابزار صدا می‌زند و پاسخ را می‌سازد.
- **لایهٔ داده و ابزار:** فهرست محصولات، موجودی زنده در سطح فروشگاه، پروفایل مشتری و تاریخچهٔ سفارش، پایگاه دانش، و جست‌وجوی بُرداری روی نقدها و دفترچه‌های راهنما.

آنچه اهمیت دارد لایهٔ میانی است. رانتایم عمومی است و کالایی شده؛ مهارت‌هایی که در آن بارگذاری می‌شوند مختص حوزهٔ خرده‌فروش‌اند و همان‌ها تخصص برند را به مشتری می‌رسانند.

نمونه‌ای از یک کتابخانهٔ مهارت

مهارت	کار	مالک	سطح
`project-guidance`	دانش فنی‌ای که پرسش مبهم («چطور حمام را کاشی کنم؟») را به برنامهٔ گام‌به‌گام بدل می‌کند: وابستگی‌های ساختاری، منطق ترتیب، و هشدار اشتباه‌های رایج.	تیم دانش فنی	فقط‌خواندنی
`materials-list`	از توصیف پروژه، فهرست گروه‌بندی‌شدهٔ مواد می‌سازد، شامل اقلامی که پیمانکار احتمالاً فراموش می‌کند.	بازرگانی حرفه‌ای	فقط‌پیش‌نویس
`review-summarize`	نقدهای طولانی محصول را به مزایا، معایب و کاربردهای رایج فشرده می‌کند.	شخصی‌سازی	فقط‌خواندنی
`delivery-window`	گزینه‌ها و زمان تحویل را با توجه به موقعیت مشتری، موجودی فروشگاه و شبکهٔ حمل حساب می‌کند.	تأمین و تحویل	فقط‌خواندنی
`return-policy`	قواعد بازگشت کالا را با ده‌ها استثنا (سفارش ویژه، مواد خطرناک، برش سفارشی، قیمت قراردادی) مستند می‌کند.	خدمات مشتری	فقط‌خواندنی

یک پرسش، چطور در کتابخانه مسیریابی می‌شود

مشتری می‌پرسد: «می‌خواهم حمام بچه‌ها را بازسازی کنم، چه چیزهایی لازم دارم؟» رانتایم در آغاز نشست فقط توصیف‌های سطح‌یک همهٔ مهارت‌ها را بارگذاری می‌کند (هر کدام حدود ۳۰ تا ۸۰ توکن). تشخیص می‌دهد که `project-guidance` می‌خواند و بدنه‌اش را بارگذاری می‌کند. اگر مشتری دربارهٔ تحویل بپرسد، مهارت `delivery-window` بارگذاری می‌شود؛ اگر دربارهٔ بازگشت کالا بپرسد، `return-policy`.

و توجه کنید چه چیزی رخ نمی‌دهد: هیچ عامل یکپارچهٔ «دستیار بازسازی» وجود ندارد که روی همهٔ سناریوهای ممکن آموزش دیده باشد. هر تکه از تخصص فقط وقتی وارد زمینه می‌شود که گفت‌وگو به آن برسد. زمینهٔ فعال کوچک می‌ماند و توانایی در دسترس عملاً بی‌کران است.

نردبان خواندن / پیش‌نویس / کنش در خرده‌فروشی

سطح	توانایی	بازبینی لازم
فقط‌خواندنی	می‌تواند داده بگیرد، پرس‌وجو کند یا توصیف کند؛ نمی‌تواند وضعیت را تغییر دهد.	تأیید تیم حوزه
فقط‌پیش‌نویس	می‌تواند محتوا برای بازبینی انسانی تولید کند؛ نمی‌تواند ارسال یا ثبت کند.	تیم حوزه + مالک قالب
مجاز به کنش	می‌تواند عملیات برگشت‌ناپذیر روی سامانه‌های واقعی اجرا کند.	تیم حوزه + امنیت و انطباق + تأیید مدیریت

نکتهٔ راهبردی مرکزی

اگر رقیب و شما هر دو روی رانتایم عمومی یکسانی کار می‌کنید — و می‌کنید، چون رانتایم کالایی شده — تطبیق رانتایم بی‌اهمیت است. **کتابخانهٔ مهارت، الگوهای انباشتهٔ شرکت را مستند می‌کند.** خرده‌فروشی که سنگین روی عامل سفارشی سرمایه‌گذاری می‌کند اما کتابخانهٔ مهارتش را رها می‌کند، روی همان بخشی از پشته سرمایه‌گذاری کرده که رقبا رایگان به آن می‌رسند.

۱۲. خودآزمایی

1. سه سطح افشای تدریجی را نام ببرید و بگویید هزینهٔ توکن هر سطح کِی پرداخت می‌شود.
2. برای مهارتی که در کلاس ساختید، سه محرک مثبت و سه محرک منفی بنویسید. اگر سخت بود، چه چیزی دربارهٔ مهارت می‌گوید؟
3. سه چیز در پروژهٔ شما که باید در `AGENTS.md` بمانند و سه چیز که باید مهارت شوند کدام‌اند؟
4. یک نمونه بیاورید که مهارت به `MCP` نیاز دارد. مرز میانشان کجاست؟
5. برای هر یک از چهار حالت شکست، بگویید کجا و چطور کشفش می‌کنید.
6. چرا ارزیابی مهارت در انزوا نتیجهٔ گمراه‌کننده می‌دهد؟
7. تفاوت `pass^1` و `pass^k` را توضیح دهید و بگویید چرا دومی برای تولید مهم‌تر است.
8. یکی از مهارت‌های شما در چه سطحی از نردبان اختیار `(Read)` است؟ برای رفتن به سطح بعد چه چیزی لازم دارید؟

9. بودجهٔ توکن پروژه‌تان را حساب کنید: یک سیستم پرامپت بزرگ در برابر کتابخانهٔ مهارت. صرفه‌جویی چقدر است؟
10. اگر امروز اجازه بدهید عامل مهارت‌های خودش را ویرایش کند، کدام سنجه را ممکن است سرش کلاه بگذارد؟

منبع: این جزوه خلاصه و بازنویسی آموزشی مقالهٔ `Agent Skills` نوشتهٔ تانوی سینگال، گابریلا هرناندز لاریوس، دبانشو دوس، لاوی نیگام و اسمیتا کولان است (مجموعهٔ مقالات عامل‌ها، می ۲۰۲۶). ترجمهٔ کامل فارسی در همین پوشه در دسترس است.

واژه‌نامهٔ این جلسه

اصطلاحاتی که در این سند به کار رفته‌اند، به‌همراه معادل انگلیسی‌شان. فهرست کامل 159 اصطلاح دوره در واژه‌نامهٔ دوره آمده است.

اصطلاح	معادل انگلیسی	توضیح
ابزار	`tool`	تابعی که عامل می‌تواند فرا بخواند تا در جهان بیرون کنش کند.
ارزیابی	`evaluation`	سنجش رفتار غیرقطعی با نمره و باند تحمل، در برابر ادعای صفر و یکی تست.
اسکیما	`schema`	ساختار تعریف‌شدهٔ داده یا ورودی ابزار.
افشای تدریجی	`Progressive disclosure`	بارگذاری سه‌سطحی: فراداده همیشه، بدنه هنگام فعال‌شدن، منابع همراه فقط در صورت نیاز.
امبدینگ	`embedding`	بازنمایی برداری متن برای جست‌وجوی معنایی.
بازیابی	`retrieval`	آوردن دانش بیرونی به زمینه در لحظهٔ نیاز.
بدهی زمینه	`Context debt`	انباشت دستورهای تأکیدی و پرحجم در توصیف‌ها که مدل یاد می‌گیرد نادیده‌شان بگیرد.
بنچمارک	`benchmark`	مجموعه‌کار استاندارد برای مقایسهٔ عملکرد.
بودجهٔ توکن	`token budget`	سقف عملی توکنی که می‌توان در زمینهٔ فعال خرج کرد.
تجزیه	`decomposition`	شکستن کاری بزرگ به زیرکارهای اتمی.
تست	`test`	وارسی قطعی رفتار کد. «یونیت تست» = آزمون واحد.
توهم	`hallucination`	تولید بااطمینانِ محتوای نادرست به‌دست مدل.
توکن	`token`	واحد کوچکی از متن که مدل پردازش می‌کند.
حافظه	`memory`	آنچه عامل میان نوبت‌ها یا نشست‌ها نگه می‌دارد.

اصطلاح	معادل انگلیسی	توضیح
حافظهٔ رویه‌ای	`Procedural memory`	به‌خاطرسپردن «چگونه انجام دادن»؛ در برابر حافظهٔ رویدادی (چه گذشت) و معنایی (چه می‌دانیم).
حالت مرزی	`edge case`	ورودی یا شرایط نامتعارف که منطق را می‌شکند.
حلقه	`loop`	چرخهٔ ادراک ← برنامه ← کنش ← مشاهده که عامل در آن می‌چرخد.
خودگردان	`autonomous`	توان اجرای کار بدون تأیید انسانی در هر گام.
دامنهٔ کران‌دار / بی‌کران	`Bounded`	ابزار در دامنهٔ کران‌دار «بفرست و فراموش کن» کار می‌کند؛ عامل در دامنهٔ بی‌کران به گفت‌وگوی چندنوبتی نیاز دارد.
داور	`judge`	مدلی که خروجی مدل دیگر را در برابر سنجه نمره می‌دهد.
داور مدل‌زبانی	`LLM-as-Judge`	مدلی هم‌رده که خروجی را در مقیاس با سنجهٔ مشخص نمره می‌دهد.
دروازه	`gate`	نقطهٔ وارسی که اجرا را تا برآورده‌شدن شرطی متوقف می‌کند.
دستور	`instruction`	قاعده‌ای که رفتار عامل را شکل می‌دهد، در برابر داده‌ای که پردازش می‌کند.
دیباگ	`debug`	یافتن و رفع علت ریشه‌ای خطا.
دیپلوی	`deploy`	انتشار کد روی محیط اجرا.
رانتایم	`runtime`	محیطی که عامل یا برنامه در آن اجرا می‌شود.
رد اجرا	`trace`	ثبت زمان‌ترتیبی گام‌های استدلال و فراخوان ابزار.

اصطلاح	معادل انگلیسی	توضیح
زمینه	`context`	اطلاعاتی که عامل در لحظه در اختیار دارد. پایهٔ «مهندسی زمینه» و «پوسیدگی زمینه».
زمینهٔ ایستا / پویا	`Static`	ایستا همیشه بارگذاری می‌شود (هویت عامل)؛ پویا فقط هنگام نیاز فراخوانی می‌شود (مهارت، نتیجهٔ ابزار، بازیابی سند).
زیرعامل	`subagent`	عاملی که عامل دیگری برای کاری مشخص فرا می‌خواند.
سنجه	`rubric`	معیاری که بر پایه‌اش به خروجی نمره داده می‌شود.
سی‌آی / سی‌دی	`CI`	یکپارچه‌سازی و استقرار پیوسته.
شکست محرک	`Trigger failure`	مهارت نادرست فعال می‌شود یا مهارت درست اصلاً فعال نمی‌شود.
عامل	`agent`	مدل به‌علاوهٔ مهار؛ توان استدلال، به‌کارگیری ابزار و اجرای کار.
فاین‌تیون	`fine-tuning`	آموزش تکمیلی مدل روی دادهٔ اختصاصی.
فراخوان ابزار	`tool call`	یک نوبت مشخص از به‌کارگیری ابزار، با آرگومان‌ها و نتیجه‌اش.
فرامهارت	`Meta-skill`	مهارتی که کارش نوشتن، ارزیابی یا بهبود مهارت‌های دیگر است.
قصد / نیت	`intent`	خواستهٔ واقعی کاربر، در برابر آنچه لفظاً گفته. ریشهٔ «انحراف قصد» و «رضایت قصد».
قطعی / غیرقطعی	`deterministic`	قطعی یعنی ورودی یکسان همیشه خروجی یکسان می‌دهد؛ مدل‌ها غیرقطعی‌اند.
قید	`constraint`	محدودیت سختی که فضای کنش عامل را کوچک می‌کند.

اصطلاح	معادل انگلیسی	توضیح
مارکت‌پلیس	`marketplace`	بازارگاه عرضهٔ عامل‌ها یا مهارت‌ها.
محرک	`trigger`	نشانه‌ای در پرامپت کاربر که باعث فعال‌شدن یک مهارت می‌شود.
مرز	`boundary`	خط جدایی میان دو ناحیهٔ اعتماد یا مسئولیت.
مسیر	`trajectory`	توالی گام‌های استدلال و فراخوان ابزار که عامل تا رسیدن به پاسخ طی می‌کند.
مسیر ابزار	`Tool trajectory`	توالی فراخوان‌های ابزار؛ آنچه عامل «انجام داد»، در برابر آنچه «گفت».
مسیریابی	`routing`	انتخاب اینکه کدام مهارت یا عامل کار را بردارد.
مشخصات	`spec`	سند قصد و رفتار مورد انتظار؛ منبع حقیقت برای عامل و انسان.
مهارت	`skill`	پوشهٔ دانش رویه‌ای که عامل فقط هنگام نیاز بارگذاری می‌کند.
مهارت عامل	`Agent Skill`	پوشه‌ای شامل SKILL.md و به‌اختیار references/، scripts/ و assets/ که دانش رویه‌ای را حمل می‌کند.
مهندسی عامل‌محور	`Agentic engineering`	همان قدرت مولد، اما درون سامانه‌ای از مشخصات، تست‌ها، ارزیابی‌ها و گاردریل‌ها با نظارت انسانی بر معماری.
نردبان اختیار	`Read`	سه سطح مجوز مهارت: فقط‌خواندنی، فقط‌پیش‌نویس، مجاز به کنش.
نشست	`session`	کل یک گفت‌وگوی چندنوبتی، از آغاز تا پایان کار.
نوبت	`turn`	یک رفت‌وبرگشت در گفت‌وگو با مدل.

اصطلاح	معادل انگلیسی	توضیح
هماهنگ‌کننده	`Orchestrator`	واگذاری ناهم‌زمان کار به چند عامل و بازبینی نتیجه؛ کار در سطح انتزاع بالاتر.
وابستگی	`dependency`	کتابخانهٔ بیرونی که پروژه به آن تکیه دارد.
واگذاری	`handoff`	سپردن کار از عاملی به عامل دیگر.
پرامپت	`prompt`	متنی که به مدل داده می‌شود. پیش‌تر «درخواست» بود که با request اشتباه می‌شد.
پس‌رفت	`regression`	خراب‌شدن چیزی که پیش‌تر کار می‌کرد.
پوسیدگی زمینه	`Context rot`	افت کیفیت مدل با بزرگ‌شدن ورودی، حتی وقتی دشواری کار ثابت است.
پوشش	`coverage`	سهمی از رفتار که آزمون یا ارزیابی واقعاً می‌سنجد.
پول‌ریکوئست	`pull request`	درخواست ادغام تغییرها در شاخهٔ اصلی.
کانتکست ویندو	`context window`	بیشینهٔ توکنی که مدل در یک نوبت می‌تواند ببیند.
کدنویسی شهودی	`Vibe coding`	توصیف خواسته به زبان طبیعی، پذیرفتن خروجی مدل و بازگرداندن پیام خطا به مدل تا وقتی کار کند؛ بدون راستی‌آزمایی نظام‌مند.
گردش‌کار	`workflow`	توالی گام‌های یک کار تکرارشونده.
گم‌شدن در میانه	`Lost in the middle`	عملکرد وقتی بهترین است که اطلاعات مرتبط ابتدا یا انتهای ورودی باشد و در میانه افت می‌کند.